팀 실험 계획: 조원 자료 통합과 과적합 대책

단일 모델 → 튜닝 → AutoGluon → 앙상블을 기기 4대에서 같은 기준으로 돌리는 계획

요약. (1) 모든 기기가 대회 양식의 고정 분할과 같은 5-fold, 같은 변수 코드를 쓴다. (2) 변수는 미리 정한 세트(F5, F5C, F5LS, F5X, FULL)와 fold 안에서 고른 세트(SEL)로 비교한다. (3) 변수를 늘리면 지금까지 **매번 CV가 나빠졌으므로**, 최종 선택은 "CV 최저 모델과 1 SE 안이면 변수가 적은 모델"로 한다. (4) 검증셋은 마지막에 한 번만 본다.

1 조원 자료를 확인한 결과

자료	확인한 것	이번 계획에서의 처리
조원 1 (압축 파일)	AutoGluon 확장(14개 변수) 테스트 RMSE 8.72, 주변 은하 변수 27개, 4성분 분해·SVM 탐색. **분할이 대회 양식과 다르다**: 그 테스트 20% 중 공식 검증셋과 겹치는 행은 20%(8,353개)뿐	점수·예측값은 공식 비교와 앙상블에 쓰지 않는다. 주변 은하 27개 변수는 이미 저장소에 있다(S 묶음, 주기 경계·전체 카탈로그로 통일). 분해(hurdle) 아이디어는 AutoGluon·스태킹으로 대체
조원 2 (인계 문서·피처사전)	변수 424개(25개 묶음), train 5-fold 검증에서 core(43개)가 RMSE 8.08로 가장 좋고 전부(234–328개)는 8.14–8.16으로 나빠짐	`sfr_features.py`를 받아 `src/`에 넣었다. 원본 13열을 뺀 411개가 `t2_` 접두사로 추가된다. T2CORE = 원본 13 + core 28 = 41개, FULL = F5X 153 + 411 = 564개. 조원 2의 SHAP 선별 89개는 학습 행 절반에서 골라 CV가 낙관적이므로 제외하고, fold 안 선택 SEL(FULL)로 대신한다

두 조원 모두 변수를 늘리면 나빠지는 결과를 얻었고, 우리 실험도 같다(4절). 과적합 우려는 실제로 확인된 문제다.

2 과적합 대책

1. **세트를 미리 정한다.** 결과를 보기 전에 정한 세트만 비교한다: F5(25, 원래 최종), F5C(47, 문헌 근거가 가장 강한 L1·L3·L4·L7), F5LS(89, 문헌 + 조원 1), F5X(153, 우리 변수 전부), FULL(우리 + 조원 2), T2CORE(조원 2 core).
2. **데이터로 고를 때는 fold 안에서만.** SEL 세트는 바깥 fold마다 학습부 안에서 Boruta 방식(각 변수를 뒤섞은 가짜 사본보다 SHAP 중요도가 커야 남김, 안쪽 3-fold 모두에서)과 중복 제거(순위 상관 0.98 초과)로 고른다. 평가 fold는 변수 선택에 관여하지 않는다.
3. **튜닝도 묶음 단위로.** 트리 튜닝에서 문헌 L1–L7과 조원 1 변수 S를 묶음별로 켜고 끄는 것을 Optuna가 nested로 고른다.
4. **규제.** 큰 세트에는 규제 LightGBM(잎 15, 최소 표본 100, 열 샘플링 0.5, L2 5)을 함께 돌린다.
5. **최종 선택 규칙(1-SE).** CV 최저 모델과 짝지은 차이가 1 SE 안인 모델 중 **입력 변수가 가장 적은** 것을 고른다. 앙상블은 가장 복잡한 것으로 친다.
6. **검증셋 1회.** 공식 검증셋은 A가 최종 선택 뒤에 한 번만 평가한다.

3 모델 실행 체계

단계	내용	세트
단일 모델	LightGBM(기본·규제), XGBoost, CatBoost(Tweedie), RF, ExtraTrees, MLP, 로그 목표 LightGBM	F5, F5C, F5LS
선형·커널·KNN	Ridge, Lasso, Tweedie GLM(스플라인), 커널 Ridge, KNN — 튜닝 없음 / nested 튜닝	F5, F5C, F5LS
딥러닝·사전학습	Tweedie MLP, TabM, RealMLP(원·로그 목표), xRFM, TabPFN v2; TabM·RealMLP nested 튜닝	F5, F5C, F5LS / 튜닝은 F5C
트리 튜닝	LightGBM·XGBoost·CatBoost nested Optuna: 모델 + 물리적 선택(Tweedie p, 반경, 문턱, 거대 은하 가중치) + 묶음 포함 여부	F5 + 선택 묶음
변수 선택	fold 안 Boruta 선택 + LightGBM·규제·XGBoost·CatBoost	SEL(F5X), SEL(FULL)
AutoGluon	변수 축: F5(완료), F5C, F5LS, F5X, FULL, T2CORE. 전처리 축(F5C): Tweedie 추가, 로그·제곱근 목표, 거대 은하 가중치, extreme_quality	표 참조
앙상블	위 모든 OOF로 가중 평균, greedy, 상위 k, 스태킹(음 아닌 선형, 작은 LightGBM). 모두 CV 안에서 학습	전부

4 지금까지 확인한 결과 (CV RMSE, 원래 최종 R0 = 7.401)

실험	CV	R0 대비
AutoGluon, F5	7.317	−0.08 (SE 0.17) — 유일하게 앞섬, 노이즈 범위
LightGBM + L7(묶인 동반 은하)	7.396	거의 같음
LightGBM + 다른 문헌 묶음 하나씩	7.51–7.62	+0.11–+0.22
LightGBM + 조원 1 변수(S)	7.562	+0.16
LightGBM + 문헌 전부 / 문헌 + 조원 1	7.94 / 7.84	+0.54 / +0.44
fold 안 Boruta 선택(153개 중 46–66개)	7.65–7.84	+0.25–+0.44 (모델 4종)
XGBoost, CatBoost, RF, MLP 단독(F5)	7.62–8.60	모두 나쁨
선형·GLM(F5)	8.70–10.0	크게 나쁨

튜닝 없는 모델에서는 변수를 늘리면 일관되게 나빠졌다. 남은 질문은 튜닝·AutoGluon·딥러닝처럼 설정이 변수에 맞춰지는 방법에서도 그런가이다.

5 기기별 작업

기기	작업	내용	시간
A 노트북	J01	단일 모델 15개(F5C 8, F5LS 7)	2.0 h
	J02	선형·커널·KNN 30개(3세트 × 5모델 × 튜닝 유무)	4.5 h
	J20	fold 안 변수 선택: SEL(F5X)(완료), SEL(FULL)	1.0 h
	J21–J22	AutoGluon F5C 로그 목표, 제곱근 목표	3.0 h
	마지막	합치기, 앙상블·스태킹, 1-SE 최종 선택, 검증셋 1회, 보고서	2 h
B Mac	J07	딥러닝·사전학습 18개(6모델 × F5, F5C, F5LS)	6.0 h
	J19	TabM·RealMLP nested 튜닝(F5C)	5.0 h
C CPU	J09–J11	LightGBM 50회·XGBoost 40회·CatBoost 25회 nested 튜닝	9.0 h
	J23	AutoGluon F5C 거대 은하 가중치	1.5 h
D GPU	J12–J15	AutoGluon F5C, F5LS, F5X, FULL, T2CORE	7.5 h
	J16–J17	AutoGluon F5C Tweedie 추가, extreme_quality	3.5 h
X 선택	J30–J31	AutoGluon F0·F3, F5LS 4시간판 (먼저 끝난 기기)	3–7 h

권장 배정: B = 조원 2(Mac), D = 조원 1(Colab GPU), C = 조원 3, A = 이 노트북.

6 실행 순서

1. 모든 기기: 같은 커밋 확인, 환경 설치, CSV 배치 → `python scripts/jobs.py run <기기>`.
2. 각 기기: `python scripts/jobs.py pack <기기>` → zip을 A에 직접 전달(공개 금지).
3. A: `jobs.py unpack` → `a04_ensemble.py` → 보고서·GitHub 갱신.